

Database Architecture & SQLAlchemy Guide

Project: Smart College Attendance Management System

1. Introduction

This document explains the data layer of the Smart College Attendance System. The project uses **Flask-SQLAlchemy** as an Object-Relational Mapper (ORM) to communicate with a **PostgreSQL** database. SQLAlchemy allows you to write Python code instead of raw SQL queries, making the application more secure and easier to maintain.

2. Core Components

A. The Database Instance (`database.py`)

The database object is initialized in a separate, lightweight file. This architectural decision prevents "circular import" errors where models and the main app try to import each other simultaneously.

```
from flask_sqlalchemy import SQLAlchemy
db = SQLAlchemy()
```

B. Application Configuration (`app.py`)

The main Flask application connects to PostgreSQL and registers the db instance.

```
# 1. Retrieve the connection string
DATABASE_URL = os.getenv(
    "DATABASE_URL",
    "postgresql+psycopg2://postgres:Biswajit2005@localhost:5432/facerec
)

# 2. Configure Flask
app.config["SQLALCHEMY_DATABASE_URI"] = DATABASE_URL
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False

# 3. Bind the database to the app
db.init_app(app)

# 4. Automatically create tables on startup
import models
with app.app_context():
    db.create_all()
```

Note: `db.create_all()` safely creates tables if they do not exist. It will not overwrite existing data.

3. Data Models (`models.py`)

Tables are defined as Python classes inheriting from `db.Model`.

The Student Table

The `Student` table is unique because it combines standard relational data with advanced PostgreSQL JSONB fields.

```
from sqlalchemy.dialects.postgresql import JSONB

class Student(db.Model):
    __tablename__ = "students"

    id = db.Column(db.Integer, primary_key=True)
    student_id = db.Column(db.String(100), unique=True, nullable=False)
    student_name = db.Column(db.String(200), nullable=False)

    # JSONB Field for AI Embeddings
    # Stores the complex facial feature arrays generated by DeepFace
    embeddings = db.Column(JSONB)

    # Helper method to serialize for the React Frontend
    def to_dict(self):
        return {
            "studentId": self.student_id,
            "studentName": self.student_name,
            # ...other fields
        }
```

Role-Based Authentication Tables

The project uses three separate tables for users: `AuthUser` (Students), `AuthTeacher`, and `AuthAdmin`. While they share common fields (email, password), splitting them ensures rigid access control boundaries and allows role-specific fields (e.g., `employee_id` for teachers).

4. How Data is Queried (Examples in Project)

SQLAlchemy translates Python methods into SQL. Here is how your project retrieves and manipulates data:

Fetching a User by Email (Sign In)

```
# SQL: SELECT * FROM auth_teachers WHERE email = 'teacher@college.edu'
user = AuthTeacher.query.filter_by(email=email).first()
```

Inserting a New Student

```
# Create a new Python object
new_student = Student(
    student_id="STU123",
    student_name="John Doe",
```

```
        email="john@college.edu"  
    )  
  
# Add to transaction and commit to PostgreSQL  
db.session.add(new_student)  
db.session.commit()
```

Updating Records

```
# Find the student, change a property, and commit  
student = Student.query.get(1)  
student.department = "Computer Science"  
db.session.commit()
```

5. Why JSONB is Used

Traditional SQL databases struggle with dynamic arrays, but PostgreSQL's JSONB dialect (imported via SQLAlchemy) bridges this gap.

- **embeddings (Student):** Stores the 512-dimensional vector array generated by the AI model.
- **students (AttendanceRecord):** Stores the list of students marked present in a specific session without needing a complex join table.